Mutual Recursion
Local Definitions
`case` Expressions
Type Variables

CSC345: Programming Languages and Paradigms

WCU
WEST CHESTER
UNIVERSITY

Mutual Recursion

- Two or more functions are defined recursively in terms of each other

Mutual Recursion

```
> evens “abcde”
“ace”
```

- *Example:* functions that select the elements at all the even and odd positions (zero-indexed)

Local Definitions

- Locals can be defined when defining a function

$$Area\ of\ triangle = \sqrt{s\ *(s-a)*(s-b)*(s-c)}, where\ s = \frac{a+b+c}{2}$$

```
triArea :: Float -> Float -> Float -> Float

triArea a b c
  = sqrt (s * (s-a) * (s-b) * (s-c))
  where
     s = (a+b+c)/2
```

```
rectArea x y
  | possible  = area
  | otherwise = 0
  where
     area     = x * y
     possible = x >= 0 && y >= 0
```

Scope

- A Haskell script consists of a sequence of definitions
- The scope of a definition is that part of a program in which the definition can be used
- Top-level definitions: scope is *entire* script that they are defined in
- Local definitions (via `where` clauses) and variables from a function definition have a more limited scope
  - *Rule:* the most local variable is the one referred to when *multiple* are in scope

```
maxsq :: Int -> Int -> Int
maxsq x y
  | sqx > sqy = sqx
  | otherwise = sqy
  where
     sqx = sq x
     sqy = sq y
     sq :: Int -> Int
     sq z = z * z
```

`case` Expressions

- General Form: *(will usually be used inside of a function definition)*

```
case e of
  p1 -> e1
  p2 -> e2
  …
  pk -> ek
```

- Match expression `e`, in turn, against the patterns p_1, p_2, …, p_k, returning the corresponding expression e_1, e_2, …, e_k
- Defining an <u>expression</u> using pattern matching, as opposed to only using pattern matching in a <u>function definition</u>

Example

```
ex01 :: Int

ex01 = case "Hello" of
  []      -> 3
  ('H':xs) -> length xs
  _       -> 7
```

Example

- A function `firstDigit` that returns the first digit in a string (wherever it occurs)

Syntactic Sugar

- Syntactic Sugar: making something in the language easier to read or express
  - A construct in a P/L is called “syntactic sugar” if it can be removed from the language without having any effect on what the language can do.
- *Prompt:* name elements of Java that are syntactic sugar

- **Big idea:** pattern matching on parameters in function definitions is actually *syntactic sugar* for case expressions

Example: defining `head'`

Type Variables

- Used to capture the idea that some specific function can be applied to arguments that have *any* type
- *Rules / Convention:* usually letters from the *beginning* of the alphabet (a,b,c,…)
  - Any identifier beginning with a small letter can be used
- Example:

```
length :: [a] -> Int
head :: [a] -> a
fst :: (a,b) -> a
```

- Other examples:

Type Variables (2)

- [Bool] -> Int is an instance of the type [a] -> Int
- [Int] -> Char is not an instance of the type [a] -> [a]

Most General Type

- in GHCi, `:type functionname` returns the most general type

```
mystery (x,y) = if x then ‘c’ else ‘d’
```

Polymorphic Functions vs. Overloaded Functions

- A polymorphic function has the *same* definition on all types, so it is the same function for each of its instances

  Example:

  ```
  fst :: (a,b) -> a
  fst (x,y) = x
  ```

- An overloaded function has *different* definitions over different types

  Example:

  == over Integer built-in

  == over pairs defined by:

  ```
  (m,n) == (p,q) = (m == p) && (n == q)
  ```